AF SECTIONS 1 TO 10

LAYER 2, JULES EXECUTION CONTRACT

Internal Use Only. Execution contract for implementation.
Version: Standalone build, validation, test, and delivery contract for intake feasibility gating.

How to use this document

This document tells Jules exactly what must be built, validated, output, and proven for AF intake.

Layer 1 governs meaning, posture, and lifecycle doctrine.
Layer 3 governs machine-bindable fields, enums, templates, conditionals, context locks, sequencing rules, and validation logic.
Layer 2 is the implementation contract between them.

Jules must not treat Layer 2 as suggestions. Locked requirements in this document are build obligations.

If a locked rule can be machine-enforced, Jules must implement that enforcement. If a locked rule cannot yet be machine-enforced, Jules must surface that explicitly in Appendix F. Jules must not silently leave a locked intake rule procedural, assumed, or operator-dependent.

Jules may not claim completion unless:

- every locked rule is either implemented or explicitly disclosed in Appendix F format
- all required tests in Appendix C are present and passing
- all required fixtures in Appendix D are present and valid
- all required delivery artifacts in Appendix E are present
- the traceability matrix is complete
- no unresolved locked-rule gap remains undocumented

SECTION 1: PURPOSE AND BOUNDARY OF INTAKE [LOCKED]

1.1 Intake as bounded feasibility gate

Jules must implement intake as a bounded feasibility gate. Intake determines whether the matter qualifies for full forensic execution under controlled browser contexts, using only the complaint or demand materials provided as the scope anchor.

1.2 Prohibited alternate implementations

Jules must not implement intake as:

- a compliance audit
- a defect inventory
- a remediation guide
- a certification
- a legal opinion

1.3 Intake output boundary

Jules must implement intake so that external-facing intake outputs express eligibility only. Intake outputs must not disclose internal depth, internal selection detail, counts, run volume, or internal sequencing.

1.4 Locked nondisclosure posture, external outputs

Any external-facing intake output must not disclose:

- number of runs performed
- number of confirmations reached
- any observed or not observed counts

- cap reached status
- which specific asserted conditions were selected as run units during intake
- per-run sequencing details
- per-run or per-asserted-condition context assignment beyond the matter-level context disclosed by the determination template or other governed external intake template

Jules must allow matter-level execution context to be disclosed using the locked

exact terms Replicated Desktop Browser Context and Replicated Mobile Browser

Context, including whether the matter was assessed under Desktop only, Desktop

and Mobile, Desktop with Mobile baseline constrained, Mobile only, or Mobile

with Desktop baseline constrained.

Jules must ensure that such disclosure remains at the matter level only. Jules

must not permit disclosure of per-run sequencing, per-asserted-condition context

assignment, or other internal execution structure.

1.5 Indirect signaling prohibition

Jules must prevent external-facing intake outputs from implying the prohibited internal details indirectly, including through phrasing such as:

- “extensive testing”

- “limited testing”
- “we checked everything”
- “we checked only a few items”
- or similar statements that signal internal execution depth

1.6 Intake is not a disguised merits workflow

Jules must not implement intake in a way that turns it into a disguised assessment, a partial audit, a remediation posture, or an external narrative about site quality, claim strength, or party conduct.

SECTION 2: AUTHORITY, SOURCE OF TRUTH, AND DRIFT CONTROL [LOCKED]

2.1 AFDM as implemented intake system

Jules must treat AFDM as the implemented automated execution system that performs intake runs and emits intake artifacts.

2.2 AFDM repository behavior as source of truth

Jules must treat AFDM repository behavior as the source of truth for:

- actual outputs
- labels
- record fields
- parameter locks
- constraint handling

Jules must not implement doctrine claims that are not supported by actual repository behavior. When Jules detects any conflict

between doctrine text and implemented AFDM behavior, Jules must not treat that conflict as permission for silent drift. Jules must surface the conflict through the governed reporting path and must not introduce new labels, fields, stop rules, procedures, or metadata classes to work around the discrepancy.

2.3 Locked internal record discipline

Jules must ensure intake records do not contain:

- evaluative fields
- intent signals
- strength indicators
- probability signals
- outcome-leaning metadata

2.4 Outcome neutrality in internal records

Jules must ensure outcome categories and record fields do not imply:

- compliance posture
- liability posture
- likelihood posture

2.5 Record terminology discipline

Jules must not frame intake entries internally as:

- findings
- violations
- assessments

2.6 Scope note

Jules must preserve the distinction that Section 2 governs internal record structure and metadata discipline. External-facing vocabulary restrictions are governed by Section 3.

2.7 Conflict rule

If doctrine text conflicts with implemented AFDM behavior, Jules must preserve the implementation as the controlling behavior for what the system actually does.

2.8 Drift control

If a conflict or ambiguity exists between doctrine text and implemented behavior, Jules must not patch the conflict by improvising new fields, labels, or procedures. Jules must surface the issue and require correction through the governed update path.

2.9 Completion blocking for unsupported doctrine

Jules may not claim an intake rule implemented if repository behavior does not actually support it.

SECTION 3: MECHANICAL OBSERVER LANGUAGE [LOCKED]

3.1 Mechanical neutrality

Jules must generate intake language that is mechanically neutral. It must record observed state under bounded execution parameters and must not characterize, evaluate, assign responsibility, imply motive, or imply intent.

3.2 Locked banned framing, external-facing intake output

Jules must prohibit the following terms in all external-facing intake output, and also prohibit functional equivalents:

- pass
- fail
- compliant

- non-compliant
- violation
- audit, as intake purpose
- remediation, as intake purpose
- certification
- guarantee
- blame posture
- adversarial posture

3.3 Definition of blame or adversarial posture

Jules must treat blame posture or adversarial posture as any language that:

- assigns fault
- implies intent
- suggests wrongdoing
- frames either party as acting improperly

3.4 Governing principle

If a term characterizes the site, the claim, or either party, rather than recording a mechanically observed state under bounded parameters, Jules must treat it as prohibited whether or not it appears on the explicit banned list.

3.5 Locked scope anchor terms, external outputs only, mandatory use

Jules must require the following exact terms in external-facing intake output when describing scope and execution context. Paraphrase is not permitted:

- “complaint or demand materials provided”
- “specific website conditions asserted in those materials”
- “bounded execution parameters”
- “Replicated Desktop Browser Context”
- “Replicated Mobile Browser Context”

3.6 Locked burden rule

Jules must preserve intake language so that it stays literal about:

- what was executed
- what was observed
- what was blocked by a constraint
- what could not be bounded due to missing specification

Jules must not implement rhetoric that shifts burden by wording.

SECTION 4: INTAKE DETERMINATION TEMPLATES [LOCKED]

4.1 One determination per matter

Jules must issue exactly one intake determination per matter.

4.2 Locked determination template set, eight options only

Jules must implement one, and only one, of the following external-facing

determination templates. No paraphrase, no alternate labels, and no additional

templates are permitted:

DETERMINATION: ELIGIBLE FOR DESKTOP AND MOBILE TECHNICAL RECORD BUILD

DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD

DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD / MOBILE BASELINE: CONSTRAINED

DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD

DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD / DESKTOP BASELINE: CONSTRAINED

DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION

DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (BOTMITIGATION)

DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (OTHER)

These eight templates are the only permitted external determinations.

4.3 Locked nondisclosure, determination line

The determination line must not reveal, directly or indirectly:

- how close intake came to sufficiency
- which asserted conditions were selected or attempted
- what was observed or not observed
- whether any run cap or stop rule was reached

The determination line must not include modifiers or hedging such as:

- provisionally
- appears
- strongly
- weakly
- likely
- unlikely
- or similar phrasing

4.4 Peer-baseline eligibility and constraint guardrails

Jules must enforce:

- Dual-baseline eligibility is expressed only through Template 1.

- Desktop-only eligibility is expressed only through Template 2.

- Desktop eligibility with a Mobile baseline constraint is expressed only through Template 3.
- Mobile-only eligibility is expressed only through Template 4.
- Mobile eligibility with a Desktop baseline constraint is expressed only through Template 5.

Template 2 is permitted only when:

- the submitted materials expressly cabin the asserted condition to Desktop only

Template 3 is permitted only when:

- Mobile baseline is in scope for the matter
- Mobile baseline execution is blocked under controlled parameters
- the internal record uses the applicable constraint_class value

Template 4 is permitted only when:

- the submitted materials expressly cabin the asserted condition to Mobile only

Template 5 is permitted only when:

- Desktop baseline is in scope for the matter
- Desktop baseline execution is blocked under controlled parameters
- the internal record uses the applicable constraint_class value

“Mobile-preferred,” “Desktop-preferred,” and “partial eligibility” determinations are prohibited.

4.5 Constraint-template selection and prohibited determination forms

Jules must enforce the following locked constraint-template selection rules:

- Template 7, DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (BOTMITIGATION), is valid only when the controlling ineligibility basis is BOTMITIGATION
- Template 8, DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (OTHER), is valid only when the matter is not eligible for forensic execution due to constraint conditions other than BOTMITIGATION.

Jules must not generate:

- partial eligibility determinations
- Template 7 for non-BOTMITIGATION ineligibility
- Template 8 when the controlling ineligibility basis is BOTMITIGATION

SECTION 5: RUN CAP, SUFFICIENCY, AND STOP RULES [LOCKED]

5.1 Run cap

Jules must enforce a maximum of 10 intake runs per matter. No time cap. No assertion-count cap. The run cap is the only bound on run volume.

5.2 Run definition

Jules must implement one run as exactly one controlled attempt of one asserted website condition under one browser context, producing exactly one outcome label from the locked outcome label set in Section 7.

5.3 Qualifying confirmations

Jules must treat only the following outcomes as qualifying confirmations:

- Observed as asserted

- Not observed as asserted

Jules must not count:

- Constrained
- Insufficiently specified for bounded execution

as qualifying confirmations.

5.4 Sufficiency threshold

Jules must implement sufficiency as reached when there are two qualifying confirmations, each produced by a distinct run, across any mix of contexts that are in scope for the matter.

5.5 Stop rule

Jules must stop intake immediately when sufficiency is reached, or when 10 runs are completed, whichever happens first.

5.6 No runs after stop

Jules must not permit additional runs after the stop condition is met for that matter, even if:

- additional asserted conditions remain
- new ideas arise
- an operator wants more coverage

5.7 Intake closure rule

A stop condition closes intake for that matter. Jules must not resume or extend intake without a new matter scope submission.

SECTION 6: COMPLAINT-ANCHORED NORMALIZATION [LOCKED]

6.1 Preserve complaint structure for traceability

Jules must preserve complaint or demand structure for traceability, while normalizing execution into atomic run units so that grouped drafting does not control operational atomics.

6.2 Locked complaint anchor forms

Jules must assign each complaint or demand grouping a COMPLAINT_GROUP_ANCHOR using one of these forms only:

- page_paragraph_range
- page_bullet_range

Jules must not invent alternative anchor types without doctrine update.

6.3 Complaint group anchor is reference only

Jules must treat COMPLAINT_GROUP_ANCHOR as a reference pointer only. It must not control how many asserted conditions are executed.

6.4 Distinct asserted website conditions become distinct run units

Jules must assign each distinct asserted website condition within a COMPLAINT_GROUP_ANCHOR its own RUN_UNIT.

A single COMPLAINT_GROUP_ANCHOR may produce multiple RUN_UNITs.

6.5 Independent run-unit execution

Jules must execute each RUN_UNIT independently and require it to yield exactly one outcome label from the locked outcome label set.

6.6 No blended testing

Jules must ensure a RUN_UNIT tests exactly one asserted website condition, with no blended testing and no multi-assertion bundling.

6.7 Locked atomicity rule

Jules must treat RUN_UNIT as the only permitted execution atomic for intake. Each intake run must map to exactly one RUN_UNIT, and each RUN_UNIT must be executed by exactly one run under exactly one browser context.

6.8 Locked traceability rule

Jules must preserve complaint anchoring for reference integrity while keeping execution atomic at the RUN_UNIT level even when the submitted materials group multiple assertions together.

Jules must not collapse multiple asserted conditions into a single RUN_UNIT for convenience or narrative symmetry.

SECTION 7: OUTCOME LABEL SET AND DEFINITIONS [LOCKED]

7.1 Locked outcome label set

Jules must support only the following outcome labels for intake runs:

- Observed as asserted
- Not observed as asserted
- Constrained
- Insufficiently specified for bounded execution

Jules must not introduce:

- additional labels
- sublabels
- qualifiers
- paraphrased variants

7.2 Observed as asserted

Jules must preserve this meaning:
Under bounded execution parameters for that RUN_UNIT and browser context, the asserted website condition manifested.

7.3 Not observed as asserted

Jules must preserve this meaning:
Under bounded execution parameters for that RUN_UNIT and browser context, the asserted website condition did not manifest.

7.4 Constrained

Jules must preserve this meaning:
Execution was blocked by an identifiable technical constraint that prevented a bounded attempt of the asserted condition, and the constraint maps to one of the locked constraint_class values defined in this doctrine.

7.5 Insufficiently specified for bounded execution

Jules must preserve this meaning:
The submitted materials do not provide sufficient bounded specification to attempt the asserted condition under controlled parameters for that RUN_UNIT and browser context.

7.6 Hard constraints on labels

Jules must not:

- replace
- collapse
- reorder
- paraphrase

the locked outcome labels.

Jules must not introduce aggregate labels, summary labels, or multi-outcome statements for a single run.

7.7 One run, one outcome

Jules must require each run to produce exactly one outcome label from the locked set.

SECTION 8: EXECUTION CONTEXT RIGOR [LOCKED]

8.1 Contexts are parameter locks

Jules must implement Replicated Desktop Browser Context and Replicated Mobile Browser Context as fixed emulation profiles. Desktop and Mobile are parameter locks, not descriptive labels.

Jules must not approximate these contexts by resizing live browser windows or development tools panels.

8.2 Replicated Desktop Browser Context, locked baseline

Jules must implement the Desktop baseline as:

- viewport: 1366 × 900 CSS pixels, fixed
- browser zoom: 100%
- deviceScaleFactor: 1

Desktop baseline is a first-class intake baseline.

8.3 Replicated Mobile Browser Context, locked baseline

Jules must implement the Mobile baseline as:

-
- viewport: 393 × 852 CSS pixels, portrait only, fixed
- orientation: Portrait, locked
- browser zoom: 100%
- deviceScaleFactor: 1
- isMobile: true

- hasTouch: true

Mobile baseline is a first-class intake baseline.

8.4 Peer-baseline scope rule

Desktop baseline and Mobile baseline are peer baselines for intake.

For generic website accessibility allegations, Jules must bring both Replicated Desktop Browser Context and Replicated Mobile Browser Context into scope under the intake determination logic.

Where the submitted materials expressly cabin the asserted condition to one baseline only, Jules must issue the corresponding single-baseline determination.

Where one baseline is feasible and the peer baseline is blocked under controlled parameters, Jules must issue the corresponding constrained-baseline determination.

Jules must not treat generic accessibility phrasing as Desktop-only by default.

8.5 Reflow separation

Jules must keep reflow testing under WCAG 1.4.10 operationally separate from baseline intake capture. Baseline viewport parameters must not be derived from reflow math.

8.6 Reflow primary method

Jules must preserve:

- viewport: 320px width
- zoom: 100%

8.7 Reflow supplemental method

Jules must preserve:

- viewport: 1280px width

- zoom: 400%
- effective width: 320 CSS pixels

8.8 Reflow does not alter baselines

Jules must not let either reflow method alter the locked Desktop or Mobile baselines.

SECTION 9: MECHANICAL NOTE DISCIPLINE AND CONSTRAINT CLASSIFICATION [LOCKED]

9.1 Purpose of notes

Jules must implement notes solely to prevent drift and preserve mechanical clarity when an attempted run cannot be bounded or is blocked. Notes must not create narrative and must not introduce evaluative framing.

9.2 Note permission gate

Jules must permit notes only when:

- outcome label is Constrained

- outcome label is Insufficiently specified for bounded execution

- the matter determination is DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD / MOBILE BASELINE: CONSTRAINED because Mobile baseline execution cannot be performed under controlled parameters when Mobile context is in scope

- the matter determination is DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD / DESKTOP BASELINE: CONSTRAINED because Desktop baseline execution cannot be performed under controlled parameters when Desktop context is in scope

9.3 Notes prohibited for

Jules must prohibit notes for:

- Observed as asserted
- Not observed as asserted
- any statement about sufficiency reached or not reached
- any statement about run ordering, context ordering, or interleaving decisions
- any attempt to explain, defend, or argue outcome selection

9.4 Mechanical sentence rule

Any permitted note must be exactly one mechanical sentence that states the blocking condition only.

9.5 Note content restrictions

Jules must ensure permitted notes contain:

- no legal terms
- no evaluative language
- no blame language
- no speculation
- no motive, severity, or probability framing

9.6 Locked constraint_class enum set

If the outcome label is Constrained, the internal record must use exactly one of the following constraint_class values:

- AUTHWALL
- BOTMITIGATION
- GEOBLOCK
- HARDCRASH
- NAVIMPEDIMENT

Jules must not create new constraint_class values.

9.7 Unmapped blocking condition rule

If a blocking condition does not map to one of the locked constraint_class values, the run must not be labeled Constrained.

Jules must instead classify the run as Insufficiently specified for bounded execution and use the mechanical note to state the missing bounded path or trigger.

SECTION 10: RUN SEQUENCING, STATE ISOLATION, AND TEMPORAL LOGGING [LOCKED]

10.1 Sequencing rule

When both Desktop and Mobile contexts are in scope for the matter, Jules must alternate contexts:
Desktop, then Mobile, then Desktop, then Mobile, continuing until a stop condition is reached.

When only one baseline is in scope for the matter, all intake runs must execute in that baseline context and no interleaving may occur.

Jules must not reorder, cluster, or otherwise manipulate run sequence to favor any asserted condition, context, or party.

When both baselines are in scope but one baseline is blocked under controlled parameters, Jules must still attempt runs in the constrained baseline in their alternating slots, record each as a distinct run using the locked outcome label set and applicable constraint_class, and count each such run against the 10-run cap.

10.2 Clean-state isolation between every run

Between every run, prior execution state must not carry forward.

10.3 Locked clean-state requirements

Jules must require:

- each run executes in a fresh, isolated browser context
- storage state does not persist between runs

If clean-state isolation cannot be established for a run, that run must not proceed to condition evaluation. Jules must classify the blocking condition using the locked outcome labels and, when applicable, the locked `constraint_class` values, and must preserve the isolation failure as internal execution metadata for that run. Clean-state isolation applies identically to Desktop and Mobile runs.

10.4 Temporal logging

Jules must record internal timestamps sufficient to show sequencing and separation, at minimum:

- run_start time
- run_end time

10.5 Temporal logging is internal only

These timestamps are internal execution metadata only. Jules must not expose them in external-facing intake output and must not reference them indirectly through phrasing such as lengthy, brief, or extensive.

APPENDIX A: LOCKED INTAKE RECORD CONTRACTS FOR LAYER 2

Layer 2 preserves these as implementation-facing record contracts. Layer 3 governs the final machine-bindable schema expression, but Layer 2 keeps the standalone field obligations Jules must implement and validate.

A1. IntakeRunRecord

Required fields:

- matter_id
- run_id
- complaint_group_anchor
- run_unit_id
- context_id
- outcome_label
- constraint_class
- mechanical_note
- run_start_local
- run_start_epoch_ms
- run_end_local
- run_end_epoch_ms

Constraints:

- one run maps to exactly one run_unit_id
- one run maps to exactly one context_id
- outcome_label must be one of the locked intake outcome labels
- constraint_class must be empty string unless outcome_label = Constrained
- mechanical_note must be empty string unless note permission gate allows a note
- complaint_group_anchor must use only the locked anchor forms
- internal timing fields are required and must not be externally surfaced

A2. ComplaintGroupAnchorRecord

Required fields:

- matter_id
- complaint_group_anchor_id
- anchor_type
- anchor_value

Constraints:

- anchor_type must be one of:

    - page_paragraph_range
    - page_bullet_range
- no alternate anchor types allowed

A3. RunUnitRecord

Required fields:

- matter_id
- run_unit_id
- complaint_group_anchor_id
- asserted_condition_text
- mobile_in_scope
- created_context_basis

Constraints:

- each run_unit_id represents exactly one asserted website condition
- one complaint group anchor may map to multiple run units
- no blended or multi-assertion run units allowed
- created_context_basis must preserve the governed basis under which each baseline was brought into scope for the matter

A4. IntakeDeterminationRecord

Required fields:

- matter_id
- determination_template
- generated_at_local
- generated_at_epoch_ms

Constraints:

- determination_template must be one of the eight locked determination templates
- exactly one determination per matter
- no paraphrase
- no modifiers
- no appended explanatory text that leaks internal execution depth

A5. IntakeManifestRecord

Required fields:

- matter_id
- scope_anchor_reference
- intake_runs
- run_units
- complaint_group_anchors
- determination_record
- internal_timing_metadata

Constraints:

- every run reference must resolve
- every run unit reference must resolve
- determination record must resolve
- internal-only metadata must remain non-external

A6. MechanicalNoteRuleRecord

Required fields:

- matter_id
- run_id
- note_permitted
- note_basis

Constraints:

- note_permitted may be true only under the locked permission gate
- note_basis must identify which locked note basis applies
- no notes for observed or not observed runs

A7. ContextProfileRecord

Required fields:

- context_id
- viewport
- orientation

- zoom
- device_scale_factor
- is_mobile
- has_touch

Constraints:

- context_id must be one of:
  - desktop_baseline
  - mobile_baseline
  - reflow_primary
  - reflow_supplemental
- baseline values must match the locked intake doctrine exactly

A8. SequencingRecord

Required fields:

- **matter_id**
- **run_sequence**
- **both_baselines_in_scope**
- **stop_basis**

Constraints:

- **when both baselines are in scope, run sequence must alternate Desktop and Mobile until stop**
- **when only one baseline is in scope, all runs must execute in that baseline context**
- **stop_basis must reflect either sufficiency reached or run cap reached**

A9. StateIsolationRecord

Required fields:

- matter_id

- run_id
- fresh_browser_context
- storage_state_persisted

Constraints:

- fresh_browser_context must be true
- storage_state_persisted must be false

A10. ExternalOutputValidationRecord

Required fields:

- matter_id
- determination_template_used
- forbidden_disclosure_check_passed
- forbidden_language_check_passed
- mandatory_term_check_passed
- matter_level_context_disclosure_check_passed
- per_run_context_leakage_check_passed
- indirect_signaling_check_passed

Constraints:

- all checks must pass for external output to be valid
- matter_level_context_disclosure_check_passed must verify that any
- context disclosure uses only the locked exact terms Replicated Desktop
- Browser Context and Replicated Mobile Browser Context at the matter level only
- per_run_context_leakage_check_passed must verify that no per-run sequencing, per-run context assignment, or per-asserted-condition context detail is present in any external-facing output
- indirect_signaling_check_passed must verify that no phrasing implies internal execution depth, run volume, or sufficiency proximity
- any prohibited language, prohibited disclosure, or per-run leakage must fail validation and block external output

---

APPENDIX B: SECTION-BY-SECTION DEFINITION OF DONE

B.1 Section 1 done when

- intake is implemented as feasibility gate only
- prohibited alternate intake purposes are blocked
- internal depth and run-count nondisclosure is enforced in external outputs
- indirect-signaling language is blocked

B.2 Section 2 done when

- AFDM behavior is treated as implementation source of truth
- internal intake records remain non-evaluative
- no improvisation path exists for doctrinal conflicts
- unsupported doctrine claims cannot be treated as implemented

B.3 Section 3 done when

- mechanical neutrality is enforced
- banned external framing is blocked
- mandatory scope anchor terms are enforced
- burden-shifting rhetoric is blocked

B.4 Section 4 done when

- exactly one determination per matter is enforced
- only eight locked determination templates are available
- no paraphrase or hedging is allowed
- peer-baseline eligibility and constraint guardrails are enforced

B.5 Section 5 done when

- 10-run cap is enforced
- qualifying confirmations are enforced correctly

- sufficiency threshold is enforced correctly, treating two distinct runs as sufficient whether they execute against the same RUN_UNIT or different RUN_UNIT values, so long as both outcomes are qualifying confirmations
- stop rule is enforced immediately
- no post-stop runs are allowed

B.6 Section 6 done when

- complaint-group anchors use only locked forms
- run-unit atomicity is enforced
- one complaint group may map to multiple run units
- no blended multi-assertion execution occurs
- all RUN_UNIT records for a matter are created before any intake runs execute for that matter, and no new RUN_UNIT is created after the stop condition has been evaluated for that matter
- RUN_UNIT records that were normalized but never executed before the stop condition fired remain in the matter record and are not deleted, relabeled, or treated as invalid because no run was executed against them

B.7 Section 7 done when

- only four locked outcome labels exist
- one run yields exactly one outcome
- label meanings are preserved
- no paraphrased or aggregate labels exist

B.8 Section 8 done when

- Desktop and Mobile baselines match locked values
- peer-baseline scope rule is enforced correctly for dual, single, and constrained baseline matters
- reflow methods remain separate from baselines
- context approximation is blocked

B.9 Section 9 done when

- note permission gate is enforced
- notes are prohibited for observed and not observed runs
- mechanical note sentence rule is enforced
- constraint_class enum is locked
- unmapped blockers are not mislabeled Constrained

B.10 Section 10 done when

- Desktop and Mobile alternation works when both baselines are in scope
- single-baseline sequencing works when only one baseline is in scope
- fresh isolated browser context is enforced every run
- storage persistence across runs is blocked
- internal timing metadata is recorded and kept internal

APPENDIX C: REQUIRED TEST MATRIX

C.1 Unit tests, required

- determination template exclusivity
- banned language detection
- mandatory scope anchor term enforcement
- qualifying confirmation logic
- sufficiency threshold logic
- stop-rule logic
- note permission gate logic
- constraint_class enum validation
- peer-baseline scope rule and single-baseline scope rule enforcement
- alternating sequencing logic
- clean-state isolation logic

C.2 Schema and record validation tests, required

- valid record acceptance for A1 through A10

- invalid record rejection for A1 through A10
- conditional note enforcement
- conditional constraint enforcement
- invalid determination template rejection
- invalid anchor-type rejection
- invalid context-profile rejection

C.3 Negative tests, required

At a minimum:

- prohibited external disclosure blocked
- indirect-signaling language blocked
- hedged determination blocked
- single-baseline determination blocked when peer-baseline scope applies and no constraint justifies the reduction
- post-stop run blocked
- blended run-unit creation blocked
- unmapped blocker mislabeled Constrained blocked
- note attached to observed or not observed run blocked
- storage persistence across runs blocked
- non-alternating sequencing blocked when Mobile is in scope

C.4 External output validation tests, required

- exact determination template preserved
- exact mandatory scope terms preserved
- banned framing rejected
- no run counts or confirmation counts leak
- no context-per-run leakage
- no “extensive” or “limited testing” style phrasing

APPENDIX D: REQUIRED FIXTURE MATRIX

Jules must provide all of the following fixtures. “Representative fixture” alone is insufficient.

D.1 Fixture 1, Desktop-only eligible matter

Purpose: proves Desktop-only determination path
Must include:

- complaint-group anchors
- run units
- Desktop-only runs
- two qualifying confirmations
  Expected state:
- Template 2

D.2 Fixture 2, Desktop and Mobile eligible matter

Purpose: proves dual-baseline eligibility

Must include:

- submitted materials with generic accessibility allegations
- alternating Desktop and Mobile runs
- two qualifying confirmations across in-scope contexts

Expected state:

- Template 1

D.3 Fixture 3, Desktop eligible with Mobile constrained

Purpose: proves Template 3 guardrail

Must include:

- Mobile baseline in scope
- Mobile baseline blocked under controlled parameters
- applicable locked constraint_class internally

Expected state:

- Template 3

D.4 Fixture 4, Mobile-only eligible matter

Purpose: proves Mobile-only determination path

Must include:

- submitted materials that expressly cabin the asserted condition to Mobile only
- Mobile-only runs
- two qualifying confirmations

Expected state:

- Template 4

D.5 Fixture 5, Mobile eligible with Desktop constrained

Purpose: proves Template 5 guardrail

Must include:

- Desktop baseline in scope
- Desktop baseline blocked under controlled parameters
- applicable locked constraint_class internally

Expected state:

- Template 5

D.6 Fixture 6, Not eligible, generic

Purpose: proves non-eligibility without constraint-specific template

Expected state:

- Template 6

D.7 Fixture 7, Not eligible, botmitigation

Purpose: proves Template 7

Expected state:

- Template 7

D.8 Fixture 8, **Not eligible, constraints other**

Purpose: proves Template 8

Expected state:

- **Template 8**

D.9 Fixture 9, **Note-gated constrained run**

Purpose: proves notes only appear when permitted

Must include:

- one Constrained run
- one mechanical note

Expected state:

- note passes validation

D.10 Fixture 10, **Invalid note on observed run**

Purpose: proves prohibited note attachment fails

Expected state:

- validation failure

D.11 Fixture 11, **Non-alternating sequencing failure**

Purpose: proves sequencing enforcement

Expected state:

- validation failure

D.12 Fixture 12, **State persistence failure**

Purpose: proves clean-state isolation enforcement

Expected state:

- validation failure

D.13 Fixture 13, Indirect-signaling output failure

Purpose: proves external wording validation rejects phrases that imply internal execution depth

Must include:

- external-facing intake output containing phrases such as extensive testing or we checked only a few items

Expected state:

- validation failure

APPENDIX E: REQUIRED DELIVERY PACKAGE

Jules must deliver the following package before claiming completion.

E.1 Change inventory

- list of changed files
- list of new files
- list of removed files, if any
- short purpose statement for each changed area

E.2 Implementation inventory

- code changes
- schema changes
- validator changes
- report generation changes
- intake run generation changes
- external output validation changes

E.3 Test inventory

- list of tests added
- list of tests updated
- evidence of passing results
- explicit note of any intentionally failing negative tests
- named CI gates
- pass/fail status for each required gate
- run identifier, job identifier, or equivalent evidence reference

E.4 Fixture inventory

- all Appendix D fixtures
- expected state for each
- validation result for each

E.5 Golden artifact verification

The delivery package must include verification output for, at minimum:

- one valid Desktop-only intake matter
- one valid Desktop and Mobile intake matter
- one valid Desktop with Mobile constrained matter
- one valid Mobile-only intake matter
- one valid Mobile with Desktop constrained matter
- one invalid external-output case
- one invalid sequencing case
- one invalid state-isolation case

For each case, show:

- fixture name
- expected state
- validation result
- reference resolution result

E.6 Doctrine-to-code traceability matrix

Jules must provide a doctrine-to-code traceability matrix with one row per locked requirement, not one row per section.

Each row must include, at minimum:

- doctrine section
- exact locked requirement summary
- implementation location
- enforcement method
- test name or identifier
- fixture name or identifier, if applicable
- limitation id, if any
- completion status

E.7 PR-ready summary

- what was built
- what remains manual-only
- what is intentionally deferred
- what risks remain
- merge blockers, if any

APPENDIX F: LIMITATIONS AND FAIL-CLOSED DISCLOSURE FORMAT

If any locked rule is not fully implemented, Jules must disclose it in this exact structure.

F.1 Required limitation entry fields

- limitation_id
- doctrine_section
- requirement_summary
- current_state, not implemented / partially implemented / manual-only
- why_not_enforced
- risk_if_unaddressed
- temporary_controls, if any
- tests_missing_or_partial
- fixtures_missing_or_partial
- expected_failure_mode, when the current state involves blocked, failing, or fail-closed behavior

- blocks_completion, true / false
- planned_followup, if any

F.2 Fail-closed rule

A limitation entry may exist only if it is explicit and traceable. Jules may not bury limitations in prose.

If a locked rule is not implemented and is not documented in Appendix F format, completion may not be claimed.

If a limitation materially affects:

- determination template integrity
- nondisclosure posture
- sufficiency logic
- stop logic
- context creation logic
- sequencing
- state isolation
- note gating
- external output validation

then blocks_completion must be true.

F.3 Completion claim rule

Jules may claim completion only when:

- all locked rules are implemented, or
- every unimplemented rule is disclosed in Appendix F format, and
- no unresolved blocks_completion = true entry remains